Code written

```
#include <assert.h>
#include <ctype.h>
#include <limits.h>
#include <math.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char* readline();
char* ltrim(char*);
char* rtrim(char*);

int parse_int(char*);

/*
* Complete the 'staircase' function below.
*
* The function accepts INTEGER n as parameter.
*/

void staircase(int n) {
for (int i = 1; i <= n; i++) {
for (int j = 0; j < n - i; j++) {
printf(" ");
}
for (int k = 0; k < i; k++) {
printf("#");
}
printf("\n");
}
```

```
}

int main()
{
int n = parse_int(ltrim(rtrim(readline())));

staircase(n);

return 0;
}

char* readline() {
size_t alloc_length = 1024;
size_t data_length = 0;

char* data = malloc(alloc_length);

while (true) {
char* cursor = data + data_length;
char* line = fgets(cursor, alloc_length - data_length, stdin);

if (!line) {
break;
}

data_length += strlen(cursor);

if (data_length < alloc_length - 1 || data[data_length - 1] == '\n') {
break;
}

alloc_length <<= 1;
```

```
data = realloc(data, alloc_length);

if (!data) {
data = '\0';

break;
}
}

if (data[data_length - 1] == '\n') {
data[data_length - 1] = '\0';

data = realloc(data, data_length);

if (!data) {
data = '\0';
}
} else {
data = realloc(data, data_length + 1);

if (!data) {
data = '\0';
} else {
data[data_length] = '\0';
}
}

return data;
}

char* ltrim(char* str) {
if (!str) {
return '\0';
}
```

```
if (!*str) {
return str;
}

while (*str != '\0' && isspace(*str)) {
str++;
}

return str;
}

char* rtrim(char* str) {
if (!str) {
return '\0';
}

if (!*str) {
return str;
}

char* end = str + strlen(str) - 1;

while (end >= str && isspace(*end)) {
end--;
}

*(end + 1) = '\0';

return str;
}

int parse_int(char* str) {
```

```
char* endptr;
int value = strtol(str, &endptr, 10);

if (endptr == str || *endptr != '\0') {
exit(EXIT_FAILURE);
}

return value;
}
```

Link

https://www.hackerrank.com/challenges/staircase/problem

